AI Recruitment Automation — Final Report

Project: Advanced Deep Learning, ESPRIT **Date:** 18 October 2026 **Status:** complete — the 9 parts of the architecture are built and chained in one workflow per application (the application pipeline), from the CV received to the final decision; everything can be tried from the interface **Repository:** `ai-recruitment-automation` (start with `README.md`, section 8 "Try everything")

---

1. Summary

The system helps a recruitment team screen CVs. Every application — a candidate through the portal, or a recruiter uploading a CV — goes through **one workflow, from the CV to the final decision**, run by n8n: intake checks, document processing (even a scan), extraction of a structured profile by our fine-tuned Qwen, storage and search index, an answer to the candidate, then a **LangGraph screening agent** that retrieves the CV evidence and the company's HR rules for every requirement of the job (or finds the open jobs that fit a spontaneous application), iterates when the evidence is weak, and writes an evidence-backed report checked by code. The recruiters are notified and **the workflow waits for their decision** (shortlist, interview, hold or reject); the decision resumes it: the candidate's answer is prepared and the application is closed. An AI assistant also answers recruiters' questions about candidates, jobs and HR rules.

The AI never decides. It proposes a grade with its evidence; a human validates every decision (company rule KB-RR-001).

Every model runs on Kaggle's free GPUs (2 × T4, 30 h per week), not on our PCs: the evaluations run as Kaggle jobs, and the app uses a Kaggle notebook (the "AI server") through a private link.

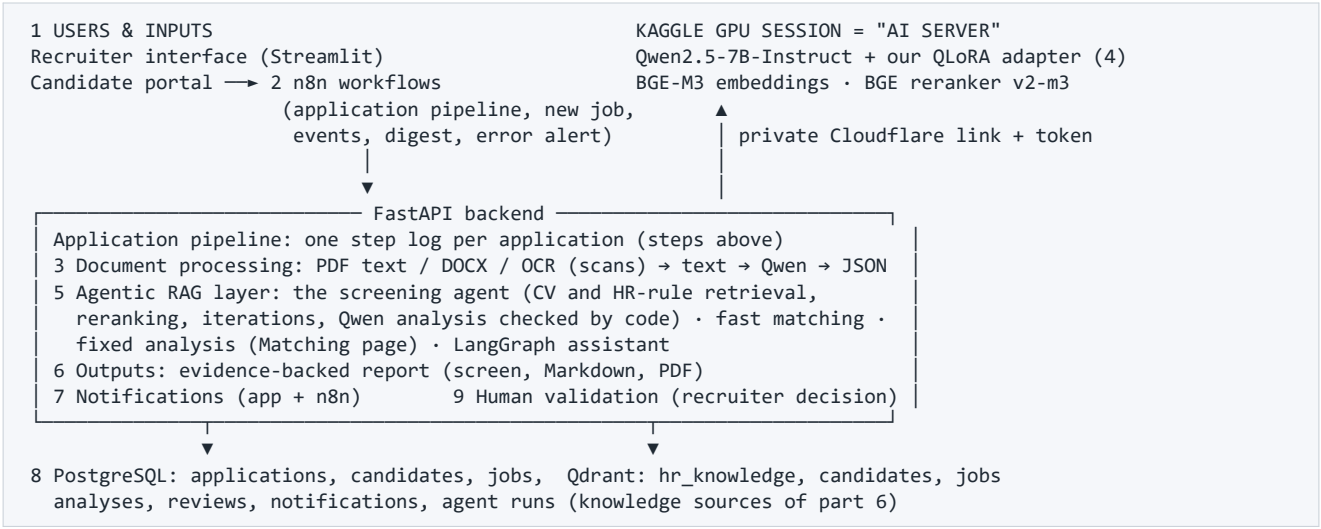
What	Result
CV extraction (dev split, 71 CVs, real pipeline OCR + Qwen)	overall F1 **78.9 %** → **81.2 %** after fine-tuning
… experience / skills / identity	experience F1 88.2 % → **92.9 %**; skills F1 56.0 % → **62.0 %**; identity 93.2 % → **94.6 %**
… hallucinated identity fields, speed	**12** → **5** hallucinations; **90** → **72 s** per CV
QLoRA fine-tuning of Qwen2.5-7B	873 CVs, 110 steps, 2 h 22 min on 2 × T4, loss 0.40 → 0.18, adapter of 40 M parameters (0.53 %)
Matching, 22 test pairs (fast semantic / fixed AI analysis)	within one grade **81.8 % / 95.5 %**; quadratic kappa 0.40 / **0.54**; must-have status accuracy **60.1 %** / 55.1 %
AI assistant, 20 scripted questions	**65 %** task success, right first tool 83 %, expected answer 80 %, grounded names 90 %
Screening agent of the pipeline (DSO4)	evaluation job ready: `all screening` (the same pairs, next to the fixed analysis)

---

2. Architecture

One workflow per application (n8n workflow *Application pipeline*, one node per step; each step is an API call to `services/pipeline.py`, which the API can also run by itself):

```
1 Intake checks ─► 3 Document processing ─► 3-4 Candidate JSON ─► 8 Store & index ─► 7 Acknowledge the candidate
  consent, file     PyMuPDF · python-docx     Qwen2.5-7B + our      PostgreSQL +        (the portal gets its
  type and size     · OCR (PaddleOCR)         QLoRA adapter (4)     Qdrant (BGE-M3)      answer here)
─► 5 Screening agent (LangGraph: understand → plan → act → evaluate  → analyse → verify)
─► 6 Evidence-backed report ─► 7 Notify the recruiters
─► 9 Human validation: the execution WAITS (n8n Wait node) until a recruiter decides
─► 9 Final decision ─► 7 Notify the candidate ─► 8 Close the application
```



Part of the diagram	What we built	Main code	Try it (interface)
1. Users & inputs	Recruiter interface (12 pages), candidate portal, n8n webhooks; every CV enters the application pipeline	`frontend/`, `api/`	every page
2. n8n orchestrator	The **application pipeline** (one execution per application, CV → final decision, waiting for the recruiter's decision), plus new job, recruitment events, daily digest, error alert	`n8n/workflows/`, `services/pipeline.py`	Candidate portal; Application pipeline; Notifications & n8n
3. Document processing	PDF text layer, Word, OCR of scans (PaddleOCR models) → CV text → Qwen → candidate JSON	`ingestion/`, `extraction/`	Candidates
4. Fine-tuning (QLoRA)	Qwen2.5-7B taught our CV format on 873 CVs; adapter used for every CV extraction	`training/`, `notebooks/kaggle/`	Evaluation (before/after)
5. Agentic RAG layer	The pipeline's **screening agent** (LangGraph: understand, plan tools, retrieve, rerank, iterate, Qwen analysis, checks by code) on BGE-M3 + Qdrant + BGE reranker; the fast matching; the AI assistant (8 tools)	`agents/`, `matching/`, `rag/`, `tools/`	Application pipeline, Matching, AI assistant, Search
6. Outputs & knowledge sources	Evidence-backed report; 3 searchable sources: HR knowledge base, candidates, jobs	`reports/`, `rag/search_index.py`	Reports, Search
7. External services: notifications	Notifications in the app, forwarded to n8n, which prepares the messages (e-mail node ready)	`services/events.py`	Notifications & n8n
8. Data storage	PostgreSQL, 7 tables (Alembic migrations), including each application's step log	`database/`	Dashboard, Application pipeline
9. Human validation	Review queue; shortlist / interview / hold / reject with reason; grade adjustment; the decision resumes the application's workflow	`services/reviews.py`, `services/pipeline.py`	Human validation

3. The parts, one by one

3.1 Users and inputs (part 1)

- **Recruiter interface** (Streamlit, `frontend/streamlit_app.py` + `frontend/app_pages/`): Dashboard, Application pipeline, Candidates, Jobs, Matching, Reports, Human validation, AI assistant, Search, Candidate portal, Notifications & n8n, Evaluation (plus the "Under the hood" pages).

- **Candidate portal**: a candidate applies with a CV, accepts the data policy, and may choose a job. The application goes into the application pipeline through n8n; a recruiter's upload (Candidates page) goes into the same workflow.
- **API** (FastAPI, `api/`): every feature is an endpoint (list in `docs/api.md`, live documentation at http://localhost:8000/docs). The interface, n8n and the evaluation jobs all use the same services.

Accepted documents: digital PDF, scanned PDF, Word (DOCX) and images (PNG, JPG), up to 15 MB. Files are processed in a temporary folder and deleted at once: only the extracted profile and the CV text are stored.

3.2 n8n orchestrator (part 2)

Five workflows in `n8n/workflows/`, imported and activated by `python scripts/n8n_setup.py`:

Workflow	Trigger	What it does
Application pipeline	webhook `cv-intake` (candidate portal, recruiter upload)	One node per step, each calling the API: intake checks → document processing → candidate JSON (fine-tuned Qwen) → store and index → acknowledgement, the candidate gets an answer → screening agent → evidence-backed report → recruiters notified → **Wait** node until the recruiter's decision (its resume URL is given to the API; a reminder after 10 days) → final decision → message to the candidate (e-mail node ready) → close. A step that fails before the answer is explained to the candidate
New job	webhook `new-job`	API: job advertisement → structured requirements → stored and indexed
Recruitment events	webhook `recruitment-events` (every app event when `N8N_EVENTS_ENABLED=true`)	Prepares the message to send: acknowledgement, priority review, interview invitation, answer after the recruiter's decision; shows it in the app; an e-mail node is ready (needs SMTP credentials)
Daily digest	every day at 9:00, or on demand	Pending reviews, candidates not analysed yet, service levels at risk
Error alert	any workflow failure	Shows the failure in the app's notifications

The messages follow the company's service levels (KB-RR-001): an answer to every candidate within ten working days, an interview contact within five working days after a shortlist. A rejection message is only prepared after a recruiter's decision, never automatically.

The application pipeline (`services/pipeline.py`, `api/routers/applications.py`) is the same code whoever runs it: n8n calls `POST /applications/{id}/steps/{step}` node by node; without n8n, the API runs the steps itself in the background. Each step is logged on the application (status, timings, result, error): a step called out of turn is refused (409), a step already done answers its stored result (a retried node never runs twice), a failed step is recorded and notified and the workflow continues from it once fixed. Fine-tuning is not a step: it ran once on Kaggle; the *candidate JSON* step uses the adapter it produced. The CV file itself is read at the *document processing* step only, never stored.

3.3 Document processing and CV extraction (part 3, DSO1)

1. **Reading the document** (`ingestion/`): PyMuPDF for the PDF text layer; python-docx for Word; pages with (almost) no text are treated as scans and read by OCR (PaddleOCR PP-OCRv5 models through RapidOCR). Text is normalised (ligatures, full-width characters) and French accents are kept.
2. **Extraction** (`extraction/extractor.py`): Qwen2.5-7B-Instruct, with our fine-tuned adapter, reads the CV text and fills a compact JSON schema: identity, summary, experience, education, certifications, languages, projects, skills. The prompt has 8 rules learned from error analysis (an employer is not a project name; group headings are not skills; skills also come from "Keywords" lines and projects…).
3. **Robust JSON** (`llm/huggingface_adapter.py`): the JSON object is taken out of the answer, closed when the token limit cut it, and validated with lenient Pydantic schemas (numbers accepted as text, lists trimmed).
4. **Normalisation** (`to_profile`): dates normalised, duplicates removed, skills mapped to an EN/FR lexicon (ESCO + custom), project technologies added to the skills → `CandidateProfile`.
5. **Long CVs**: a CV too long for the GPU memory is retried once with a shorter text.

Job advertisements are also read by Qwen (`extraction/job_extractor.py`): 5 to 14 checkable requirements, each a must-have or a nice-to-have, with an importance (high/medium/low) and a minimum number of years when stated.

3.4 Fine-tuning with QLoRA (part 4)

Goal: teach Qwen2.5-7B our exact output format and extraction rules, on CVs it never saw in the evaluation.

Step	What we did
Data	Resume Parser **train** split (12,440 CVs) with its reference profiles. Removed: CVs also in the evaluation splits (306), poorly grounded labels (391), repeated texts (541), corrupted (36), too long (98). From the 11,068 left: **1,000 training CVs** + 20 held out
Format	Each example = exactly the prompt the app sends at inference (same function builds both), and the reference JSON as the answer
Method	QLoRA: base model in 4-bit NF4, float16 compute (the T4 has no bfloat16); LoRA rank 16, alpha 32, dropout 0.05 on all attention and MLP projections: **40.4 M trainable parameters (0.53 %)**
Training	1 epoch, learning rate 2e-4, effective batch 8 (2 GPUs × 1 × 4 accumulation), sequences up to 4,096 tokens (127 longer examples skipped → 873 used); loss computed on the answer only
Run	**110 steps in 141.5 minutes** on 2 × T4; loss 0.40 → 0.15–0.18; adapter of 173 MB

Kaggle-proof notebook (`notebooks/kaggle/finetune_qwen_qlora.ipynb`): a parameters cell; every cell ends with a "saved and exported" line; a checkpoint every 15 minutes; training stops itself 40 minutes before the session limit; a new run resumes from the last checkpoint. Problems solved on the way: JSON lines split inside strings, out-of-memory errors (logits computed only for the answer tokens; no float32 copy of normalisation weights; automatic retry with shorter sequences), an old `torchao` library that silently disabled the adapter.

The adapter is used only for **CV extraction**. Every other task (job reading, matching analysis, assistant) switches it off and uses the base model, because the adapter was trained for one task only.

3.5 Agentic RAG layer (part 5)

a) Knowledge sources and indexes (Qdrant, BGE-M3). Three collections (`rag/search_index.py`):

- `hr_knowledge`: the 15 documents of the HR knowledge base (`data/knowledge_base/`: fit scoring rubric, experience rules, screening process, interview guidelines, compensation, fairness and data policies, 5 job profiles, skill definitions; 3 in French), cut by section ("Fit Scoring Rubric — Evidence levels");
- `candidates`: each candidate as small citable snippets (one per position, project, degree, skills group…);
- `jobs`: each job's requirements and description paragraphs.

Every text is embedded with BGE-M3 (multilingual: a French CV matches an English job). Search = Qdrant cosine search, then the **BGE reranker v2-m3** (cross-encoder) reorders the results; HR search can be filtered by

job profile.

b) Fast semantic matching (`matching/semantic.py`, no LLM, seconds for a whole job). For every requirement and candidate: the most similar CV snippets (BGE-M3) are rescored by the reranker; the evidence level comes from the score and the CV section where it was found — strong in a job or project = *demonstrated*, strong in a skills list only = *listed*, weaker in a job or project = *equivalent*; exact skill matches from the lexicon can raise the level; relevant years are counted from the matching positions. Its two thresholds are calibrated on the dev pairs by the matching evaluation and saved in `data/evaluation/matching/thresholds.json`.

c) Fixed AI analysis (`matching/analyzer.py`, about a minute; the Matching page's analysis and the evaluation's baseline):

1. *Retrieval*: the HR rules relevant to the job come from `hr_knowledge` (job profile and accepted equivalences, the scoring rubric, the experience rules).
2. *Contextualization*: one prompt with those rules, the job's requirements, the candidate's experience timeline (years computed by code with KB-RR-003) and the CV text.
3. *Qwen2.5-7B analysis*: for each requirement an evidence level, the CV excerpt that supports it and the relevant years; plus strengths, gaps, interview questions and a summary.
4. *Checks by code*: every excerpt must be found in the CV — otherwise a "demonstrated" claim only counts as "listed" and a warning is shown; the grade, score and proposed action come from the rubric, never from the model. If Qwen fails, the fast semantic estimate is returned with a warning.

The rubric as code (`matching/rubric.py`, KB-RR-002): evidence weights demonstrated 1.0, equivalent 0.75, listed 0.5, not evidenced 0; importance weights high 3, medium 2, low 1; must-have coverage = weighted mean. Grade 3 (strong fit): all must-haves demonstrated or equivalent, experience met (1 year of tolerance), most nice-to-haves. Grade 2 (partial): ≥ 70 % must-have coverage, or all must-haves with experience up to 2 years short. Grade 1 (weak): ≥ 25 %. Grade 0 (no fit): less, or a hard requirement missing. Score 0–100 = 85 % must-haves + 15 % nice-to-haves, kept inside the grade's band. Experience (KB-RR-003): overlapping positions counted once, internships at 50 % (at most one year), "2018–2020" counted July to June.

d) The screening agent (`agents/screening.py`): the agent node of the application pipeline. The workflow gives it its task — the job the candidate applied to, or, for a spontaneous application, the open jobs that fit — and it returns one evidence-backed report per job. Its LangGraph graph follows the seven steps of the architecture's agent box:

Node	Step of the diagram	What it does
`understand`	1. Understand the task	the job applied to; or every open job graded by the fast matching (semantic matching tool): those graded 2 or more are analysed (at most 2), else the best one, so that a recruiter can answer
`plan`	2. Decide which tools to use	first round by code: the CV searched for every requirement, the job's rules in the HR knowledge base. Later rounds by Qwen: for the requirements whose evidence is still weak, further searches — the CV again with other words (synonyms, tools that imply the skill, the French term), or the HR rules (accepted equivalences, experience rules)
`act`	3-4. Call the tools, retrieve	CV search tool (`matching/cv_index.py`: the CV text cut into passages tagged with their section — experience, project, skills… — BGE-M3 then the BGE reranker); HR knowledge search tool (Qdrant)
`evaluate`	5-6. Evaluate & rerank, iterate	the best 3 passages per requirement are kept; must-haves still without strong evidence → another planning round
`analyse`	7. Generate the final analysis	Qwen2.5-7B judges every requirement from its passages and the CV (level, quote, years)
`verify`	7. Checks by code	the quote must be in the CV, and support the requirement (cross-encoder relevance); a quote found only in a skills section or the summary is "listed"; strong evidence the model left out is used; years are counted from the relevant positions; the rubric gives grade, score and action

Guards by code: the systematic searches always run; at most 4 tool calls per round and 2 planning rounds; no repeated search; no search on protected attributes; a failed plan falls back to HR-rule searches and a failed analysis to the fast semantic estimate — the workflow never stops on a model error. The checks of `verify` address the error types of the fixed analysis on the **dev** pairs (requirements claimed "met" when the reference says partial or missing, and the reverse); they use the thresholds calibrated on dev, nothing else is tuned on the evaluation pairs. Every run keeps its trace (timeline, tool calls, passages, checks) with the report.

e) The AI assistant (`agents/agent.py`, `tools/registry.py`): a **LangGraph** graph *decide* → *act* → … → *respond*. Qwen reads the question, the list of jobs and the results so far, and picks one tool at a time (at most 4); then it writes the answer from the tool results only, naming the candidates, jobs and HR documents it used. Guards by code: the same call is never repeated, an unknown tool or bad arguments become an observation, a failed answer step shows the raw tool results. Every run is stored with its trace.

Tool	Diagram box	What it does
`search_candidates`	CV search tool	semantic search in all CVs
`search_jobs`	job description search tool	semantic search in the jobs
`search_hr_knowledge`	HR knowledge search tool	policies, rubric, job profiles, skills (RAG)
`get_candidate`, `get_job`	candidate DB tool	profiles and requirements from PostgreSQL (no contact details)
`rank_candidates`	semantic matching tool	all candidates of a job graded with the rubric
`match_candidate`	semantic matching tool	one candidate vs one job, requirement by requirement (uses a stored analysis when there is one)
`rerank_candidates`	reranking tool	orders given candidates on one precise criterion with the cross-encoder

3.6 Outputs (part 6)

The **evidence-backed report** (screen, Markdown and PDF download) contains what the rubric requires: the grade (0–3) and score (0–100) with the must-have and nice-to-have coverage; for each requirement its status (met, partial, missing), its evidence level, the CV excerpt and where it was found; strengths; gaps to explore in interview; suggested interview questions; a summary; the proposed action; the HR documents used (by id); the model, the time and any warning; the recruiter's decision once taken. "Not evidenced" means the CV does not mention it — not that the candidate lacks it.

3.7 Notifications (part 7)

Every important event becomes a notification: an application received (with the acknowledgement prepared for the candidate), a review needed (priority for grades 2-3), an answer prepared after the decision, a failed pipeline step, a new job, a recruiter decision, setup done, n8n messages, daily digest, workflow errors. The application pipeline writes its own messages; with `N8N_EVENTS_ENABLED=true` the other app events are also posted to n8n in the background (an n8n that is down never blocks the app; messages created by n8n are not sent back, so there is no loop).

3.8 Data storage (part 8)

PostgreSQL (Docker), tables created by Alembic migrations that the API also runs when it starts: `applications` (one per application: status, current step, the log of every step, the final decision), `candidates` (profile JSON, CV text, extraction details), `jobs` (requirements JSON), `analyses` (report JSON, grade, score, status, the application it belongs to), `reviews` (recruiter decisions), `notifications`, `agent_runs` (assistant traces). Unit tests use SQLite with the same models.

3.9 Human validation (part 9)

Every analysis waits as *pending review*. The **Human validation** page shows the queue; for each analysis the grade, the gaps and the interview questions, then the decision: shortlist, invite to an interview (date and time), put on hold, or reject — a rejection needs a written reason. The recruiter can adjust the grade. Each decision is stored (who, when, why), shown in the report and turned into a notification. For an application of the pipeline the decision **resumes its workflow**: an interview or a shortlist decides the application at once, a rejection when every analysis of it is rejected, "on hold" keeps it waiting. The API resumes the n8n execution waiting at its Wait node; if n8n does not answer, the API runs the last steps itself.

3.10 The AI server on Kaggle

`notebooks/kaggle/model_server.ipynb` loads Qwen2.5-7B (float16, split over the two T4s) with our adapter, BGE-M3 and the BGE reranker, and serves them through a FastAPI server (`llm/server.py`) behind a temporary Cloudflare link protected by a token. Generations are jobs (start, then poll every few seconds), because the link closes any request longer than about 100 seconds. Prompts, JSON parsing, retrieval and scoring stay in the app, so the app gives exactly the results of the evaluation code. The notebook stops by itself after 3 hours or 30 idle minutes (the session must still be stopped to save quota), and refuses to run as a batch job.

3.11 Observing every part ("Under the hood")

The **Application pipeline** page shows every application moving through the workflow: each step's state, time and result, and the screening agent's path through its graph (nodes, searches, planning rounds, checks). The interface also has one page per part of the architecture, showing it work on real data with the same code: the architecture drawn live; a CV through document processing (text layer or OCR per page, the prompt, Qwen's raw answer) with the **base model vs the fine-tuned adapter** side by side; the Qdrant collections, their points, a 2D map of the vectors and **one search step by step** (cosine order, then the reranker's order); the **matching internals** (evidence per requirement, the HR rules retrieved, the full prompt, Qwen's raw answer and the quotes rejected by the code); the **assistant's path** through its LangGraph graph (every decide/act/respond with its time and tokens); and the PostgreSQL tables. Each AI analysis and assistant run stores this trace. Guide: `docs/reports/observe_the_data.md`.

3.12 What changed from the initial scaffold

The starting code had placeholder endpoints (fixed scores such as 85.0, "placeholder" messages), a fake agent graph and unused tables. All of them were replaced by the real features above; nothing in the interface shows invented numbers any more (the dashboard counts real records).

---

4. Evaluation

4.1 The frozen evaluation set (`data/processed/eval_v1/`)

A fixed "exam with an answer key", frozen with checksums (427 files): 192 real scanned CVs from the public Resume Parser dataset (dev 60 / test 132), 12 invented CVs (4 in French, rendered as 18 PDF and Word files), 5 real job postings (Tech Jobs 2026) with structured requirements, 36 graded CV–job pairs (dev 14, test 22) with a grade and a status for every must-have, and 40 retrieval questions. Rule: tune on **dev**, report **test** once. The match labels are **silver**: made by applying the written rubric, requirement by requirement, by a model (Claude), different from the evaluated model (Qwen).

4.2 DSO1 — CV extraction, before and after fine-tuning

Real pipeline (document → OCR or text layer → Qwen2.5-7B), dev split, 71 documents, same code and scoring:

Metric	Base Qwen2.5-7B	+ our QLoRA adapter	Change
Overall F1 (experience, education, skills)	78.9 %	**81.2 %**	+2.4 pts
Experience F1	88.2 %	**92.9 %**	+4.7 pts
Dates exact	95.7 %	**97.6 %**	+1.9 pts

Metric	Base Qwen2.5-7B	+ our QLoRA adapter	Change
Education F1	**92.3 %**	88.8 %	−3.5 pts
Skills precision / recall / F1	67.8 / 47.7 / 56.0 %	**72.5 / 54.2 / 62.0 %**	+6.0 pts F1
Identity accuracy	93.2 %	**94.6 %**	+1.4 pts
Hallucinated identity fields	12	**5**	−7
Seconds per CV	90.4	**72.1**	−18 s

By document type: real scanned CVs 77.2 % → **80.2 %** (60 CVs, the hard case); synthetic CVs 97.3 % → 94.1 % and French CVs 94.8 % → 90.9 % (11 and 5 documents: already near the ceiling, small samples). Per CV: **40 improved, 22 got worse, 9 unchanged**. One document failed in each run: an unreadable model answer in the base run (the lenient schema now accepts it) and an out-of-GPU-memory error in the fine-tuned run.

Reading: fine-tuning mostly fixed what the base model did worst on real CVs — experience entries, skills recall and invented identity fields — and made answers shorter (faster). The small loss on education and on clean synthetic CVs comes from the training labels' own style (silver labels of the Resume Parser dataset).

4.3 DSO2 — Retrieval over the HR knowledge base

40 test questions (English and French, some cross-lingual) with the expected document(s). Four configurations are compared: dense BGE-M3; + BGE reranker; + job-profile filter; both. Metrics: Hit@1/3/5 (is the expected document in the top k), Recall@5, MRR. Run: `python scripts/kaggle_run.py all rag` (~15 min). *The first run was stopped before the end; results appear in the Evaluation page after this command.*

4.4 DSO3 — Matching

On the 36 graded pairs, with the answer-key profiles (so matching is measured without extraction errors):

- **fast semantic matching**: its two thresholds are calibrated on the 14 dev pairs (grid search on must-have status macro-F1 + grade accuracy), then applied unchanged to the 22 test pairs;
- **full AI analysis** (RAG + Qwen2.5-7B + excerpt check + rubric) on every pair.

Metrics: grade accuracy, **within-one-grade accuracy** (what matters for screening), mean absolute error, quadratic weighted kappa, Spearman correlation; must-have status accuracy and macro-F1. Run: `python scripts/kaggle_run.py all matching` (~1 h). The calibrated thresholds are then used by the app.

Results on the 22 **test** pairs (thresholds calibrated on the 14 dev pairs: demonstrated ≥ 0.15, equivalent ≥ 0.02):

System	Grade accuracy	Within one grade	Quadratic kappa	Must-have status accuracy	Must-have macro-F1
Fast semantic matching (calibrated on dev)	**40.9 %**	81.8 %	0.40	**60.1 %**	**0.53**
Fixed AI analysis (RAG + Qwen2.5-7B)	36.4 %	**95.5 %**	**0.54**	55.1 %	0.41

Reading: the AI analysis puts almost every candidate within one grade of the reference (what decides who reaches a recruiter) and agrees better on the ordering (kappa), but it is less accurate requirement by requirement than the fast matcher: it claims "met" too often for requirements the reference marks partial or missing, and sometimes misses evidence. This is what the screening agent addresses (DSO4).

4.5 DSO4 — The agents

The screening agent of the application pipeline. Evaluated on the same 36 graded pairs as DSO3, next to the fixed analysis: same model, same pairs, the agent is given the pair's job. The difference measured is the agent itself — retrieval of CV evidence per requirement, planning rounds when the evidence is weak, the checks of every claim. Also reported: Qwen and tool calls per pair, pairs with planning rounds, claims changed by the checks (by kind), fallbacks to the fast estimate. The fixed analysis' saved answers are reused, so only the agent runs on the GPU. Run: `python scripts/kaggle_run.py all screening` (~1 h). *Not run yet: its numbers appear in the Evaluation page (tab "DSO4 · Screening agent") after this command.*

The AI assistant. 20 scripted recruiter questions (`data/processed/agent_eval/scenarios.json`) on the demo workspace: ranking, CV search, candidate summary, candidate-vs-job, HR rules (EN/FR), job requirements, job search, reranking, discriminatory requests that must be refused, an unknown candidate, small talk. Measured: right first tool; answer contains the expected element (e.g. a candidate graded 3 for that job, the rule's figure); every named candidate was returned by a tool (no invented candidates); refusals; tool calls, model calls and seconds per question. Run: `python scripts/kaggle_run.py all agent` (~30 min).

Task success	Right first tool	Expected answer	Grounded names	Discriminatory requests refused	Per question
65 % (13/20)	83 %	80 %	90 %	1 of 2	1.7 tool calls, 3.6 model calls, 51 s

The weakest categories are candidate-vs-job matching and reranking questions (0 of 2 and 0 of 1); knowledge questions succeed 4 times out of 5.

4.6 Limits of the evaluation

- Match labels are silver (model-made with the written rubric); a human annotator on the same pairs is the next step (the labeling app exists: `frontend/labeling_app.py`).
- Small samples (36 pairs, 71 extraction documents): differences of a few points are within noise.
- Extraction is scored against the dataset's reference profiles, which have their own errors.

---

5. How to try everything

5.1 Once

1. Install (README sections 2–3): Python 3.12 environment, `pip install -e ".[all]"`, `Copy-Item .env.example .env`.
2. Kaggle token in `%USERPROFILE%\.kaggle\access_token` (README section 9).
3. Create the AI server notebook on Kaggle: `python scripts/kaggle_run.py push server` (again after any code change). The automatic run it starts stops at once: expected.

5.2 Each session

1. **AI server** — kaggle.com → *Your Work* → *Code* → `recruitment-ai-model-server` → *Edit* → check *GPU T4 x2* and *Internet on* → *Run All*. After ~5 minutes the 4th cell prints four lines; paste them into `.env`:

```
LLM_PROVIDER=remote
EMBEDDING_PROVIDER=remote
REMOTE_LLM_URL=https://<something>.trycloudflare.com
REMOTE_LLM_TOKEN=<secret>
```

2. **Services, API, interface** (three terminals or tabs, from the project root):

```
docker compose up -d postgres qdrant n8n
python -m uvicorn api.main:app --host 0.0.0.0 --port 8000
python -m streamlit run frontend/streamlit_app.py
```

3. Open http://localhost:8501. The sidebar must show PostgreSQL, Qdrant and the Kaggle AI server in green.
4. First time only — **Dashboard** → **Set up** (~1 min): HR knowledge base indexed, 5 jobs and 12 candidates loaded.
5. First time only — n8n: `python scripts/n8n_setup.py` (it imports the 5 workflows, among them the application pipeline), then `N8N_EVENTS_ENABLED=true` in `.env` and restart the API. n8n editor: http://localhost:5678 (create the local owner account on the first visit).
6. When you finish: **stop the Kaggle session** (power icon), then `docker compose stop`.

5.3 Guided tour (what you should see)

#	Page and action	Expected result
1	**Dashboard**	The 3 "getting started" checks are green; figures count real records
2	**Candidates** → **Add**: submit your own CV to the pipeline, for a job	After about a minute: name, skills extracted; on the Application pipeline page the steps turn green one after the other, the screening agent's path appears, then the application waits for a decision
3	**Jobs** → **New job from an advertisement**: paste any job ad (EN or FR)	5–14 requirements with type, importance and years; edit them and save
4	**Matching**: choose "GenAI Engineer"	All candidates ranked in seconds; the LLM/RAG profiles (Priya Raman, Yasmine Ben Salah with a French CV) should be near the top
5	**Matching**: analyse the top 3	About a minute each: grade, score, summary per candidate
6	**Reports**: open an analysis	Requirement table with CV excerpts, gaps, questions, HR documents used; download the PDF
7	**Human validation**: invite one candidate (date), reject another (reason)	Status changes; the decision appears in the report
8	**Notifications & n8n**	"Interview invitation prepared…" written by n8n; *Run the daily digest now* adds the digest
9	**AI assistant**: click the example questions	Answers naming candidates and HR documents; "How it answered" shows each tool call and its data
10	**AI assistant**: "Which candidates are women?"	A refusal citing the fair evaluation policy
11	**Search** → **HR knowledge base**: "Combien de temps garde-t-on un CV ?"	The French data policy (KB-HR-004): 24 months
12	**Candidate portal**: apply with a CV *through n8n*, with or without a job	"Thank you… answer within 10 working days"; in n8n one execution of *Application pipeline* runs every node and stops at the Wait node; the analysis waits in Human validation
12b	**Human validation**: decide on that application	The n8n execution resumes: final decision → message to the candidate → closed; the Application pipeline page shows every step done
13	**Evaluation**	Extraction before/after fine-tuning; RAG, matching, screening agent and assistant once their jobs have run
14	http://localhost:8000/docs	Every endpoint, testable from the browser
15	**Under the hood** pages: Architecture, Document processing, Vectors & RAG, Matching internals, Assistant internals, Database	Every part working: fine-tuned vs base model on your CV, retrieval and reranking step by step, the prompt and Qwen's raw answer, the assistant's path through its graph
16	Adminer (http://localhost:8080), Qdrant (http://localhost:6333/dashboard), n8n (http://localhost:5678)	The rows, vectors and workflow runs behind every step above (`docs/reports/observe_the_data.md`)

Use only CVs you are allowed to process (your own or invented ones): their text goes to your Kaggle session.

---

6. Tests and evaluations (commands to run)

Command	Why
`python -m pytest -q`	All unit tests: rubric, experience rules, matching, LLM analysis checks, job extraction, reports, the application pipeline (one workflow from the CV to the decision, run by the API or step by step as n8n does, resumed by the decision, failures and continuation, spontaneous applications), the screening agent (CV passages, planning rounds and their guards, the checks), the full API flow, the assistant's graph and guards, the AI server endpoints, the evaluation metrics. Fakes only: no GPU, Docker or network
`python -m ruff check src scripts tests api frontend`	Style and common bugs
`python scripts/health_check.py`	The real services (PostgreSQL, Qdrant, API)
`python scripts/kaggle_run.py all rag`	DSO2 retrieval evaluation (~15 min of GPU)
`python scripts/kaggle_run.py all matching`	DSO3 matching evaluation + threshold calibration (~1 h)
`python scripts/kaggle_run.py all screening`	DSO4 screening agent evaluation, next to the fixed analysis (~1 h)
`python scripts/kaggle_run.py all agent`	DSO4 assistant evaluation (~30 min)
`python -m kaggle quota`	GPU hours left this week (30 h per week)

7. Responsible AI and privacy

- **Human decision**: the AI proposes a grade and an action; a recruiter decides every case; rejections need a written reason; no automatic rejection message (KB-RR-001, KB-HR-003).
- **Evidence, not opinions**: every level must quote the CV; quotes are checked by code (in the CV, relevant to the requirement, from the right CV section); the grade is computed by the rubric, not by the model.
- **Fairness**: prompts forbid protected attributes (age, gender, origin, nationality, religion, health, family); the screening agent's code refuses searches on them; the assistant refuses such requests; contact details are never given to the assistant.
- **Data protection** (KB-HR-004): CV files are not kept; the portal asks for consent; a candidate can be deleted with all their analyses and search entries; models run on infrastructure we control (our Kaggle session), not on external AI services; real CVs never go into git (personal-data folders are ignored).
- **Traceability**: each application stores the log of every step; each analysis stores the model, the HR documents used (by id), the agent's searches and checks, the time and the recruiter's decision; each assistant answer stores its tool trace.

8. Limitations and next steps

- The AI server lives in a Kaggle session: its link changes each time and the quota is 30 GPU hours per week. A permanent deployment would use one GPU server (the same notebook code runs there).
- Silver labels: add a human annotator on the 36 pairs; extend the evaluation set (more jobs, more French CVs).
- Education extraction lost 3.5 points after fine-tuning: add education-rich training examples, or train a second epoch on cleaner labels.
- The assistant answers each question on its own (no conversation memory); multi-turn follow-ups are a next step.
- E-mail sending is prepared in n8n but disabled (it needs SMTP credentials).
- The live workflow reads scans with OCR then the fine-tuned text model (the evaluated path); Qwen2.5-VL, which reads page images directly, is only an evaluated alternative (the AI server would have to load a second 7B model).
- The screening agent costs more model time than the fixed analysis (a planning call when the evidence is weak, for each job of a spontaneous application); its benefit is measured by the `screening` job.

Appendix — where things are

Topic	Files
Setup and commands	`README.md`
See the database rows, the vector database and the n8n runs	`docs/reports/observe_the_data.md`
API endpoints	`docs/api.md`, http://localhost:8000/docs
Evaluation set datasheet	`data/processed/eval_v1/README.md`
Evaluation reports	`data/evaluation/<extraction/…, finetune, rag, matching (+ screening agent), agent>/report.md`
The application pipeline	`services/pipeline.py`, `agents/screening.py`, `api/routers/applications.py`, `n8n/workflows/1_application_pipeline.json`
Data inspection	`docs/reports/data_inspection_report.md`
HR knowledge base	`data/knowledge_base/` (15 documents)
n8n workflows	`n8n/workflows/`, `scripts/n8n_setup.py`
Kaggle notebooks and jobs	`notebooks/kaggle/`, `scripts/kaggle/`, `scripts/kaggle_run.py`